RH850 串列 ISP 工程配置

MCU 串列 ISP 工程配置

本章的UART / Xmodem / I2C各使用獨立Boot / App / Transfer工程，並保留獨立文字Log。 Flash、IRQ與RAM執行細節集中於 移植與Flash。

RH850 文件入口

目的	說明位置
Classical CAN / CAN FD、CAN port 與接收資源	CAN device 工程
UART / Xmodem / I2C、獨立 Log 與串列主迴圈	本文件
Tool 操作、完成提示與更新報告	串列操作、共用操作
格式欄位與通訊封包	串列工程配置；實際地址與 IRQ 使用本文件配置

各介面都有獨立 Boot / App / Transfer 工程。 共用格式與操作透過上方連結查閱，平台地址、腳位與中斷配置集中在各自工程章節。

配置與資料來源

專案與目錄

下列 Boot / App / Transfer 各自包含來源碼、設定與建置工具，可獨立編譯。 每個介面使用自己的工程與映像契約；搭配安裝的 Boot / App 保持相容設定。 目錄與使用者入口見 工程目錄與程式入口。

介面	Boot	App	Custom Transfer
UART ISP	Sample_Project_RH850_S1_ISP_Boot_loader_UART	Boot 名稱加 _app	Sample_Project_RH850_S1_UART_Transfer
XMODEM-CRC128	Sample_Project_RH850_S1_ISP_Boot_loader_Xmodem	Boot 名稱加 _app	Sample_Project_RH850_S1_Xmodem_Transfer
I2C ISP	Sample_Project_RH850_S1_ISP_Boot_loader_I2C	Boot 名稱加 _app	Sample_Project_RH850_S1_I2C_Slave_Transfer

Transfer 是以燒錄器安裝的獨立診斷程式，不是 App 更新 BIN。 診斷路徑不擦寫 Flash；Xmodem Transfer 使用 command24 接收串流並核對 CRC32。

接線與時脈

用途	MCU 周邊 / 腳位	設定	EIINT channel
UART / Xmodem 資料	RLIN31 TX P0_5、RX P0_4	115200、8-N-1	RX122、Error123
I2C 資料	RIIC1 SCL P8_1、SDA P8_0	7-bit address0x20、100kHz	TX240、Event241、RX242；TEI243 保持遮罩
獨立文字 Log	RLIN30 TX P10_10、RX P10_9	115200、8-N-1；接收 IRQ 遮罩	不使用資料介面接收
系統 tick	OSTM0	1ms	84
系統時脈	外部16MHz / PLL	CPU120MHz	App 檢查 Boot 交接時脈

UART 與 I2C 腳位依選定 SDK 範例的初始化程式配置。 連到 bridge 前仍需核對 EVM 接頭、電壓、共地與 I2C 上拉。 診斷 UART 不接到二進位 ISP 資料鏈路。

文字 Log 使用專案內 `retarget.c` / `retarget.h` 的 `tiny_printf()`，由 `SendChar()` 輸出至 UART0。 格式與使用位置見 App 執行標記與 counter。

UART建置以 `validate_serial_settings.ps1` 比對INFO / metadata baud與該工程SC產生的採樣、prescaler及LBRP。 執行時另核對實際UART暫存器計算的baud誤差，EVM的RLIN source為40MHz。 變更速率時同步調整配對Boot / App的SC與 `isp_transport_config.h`；不變更獨立文字Log設定。

編輯、產生與建置

建置與檔案使用

在任一工程目錄以 PowerShell 執行：

```
.\build.ps1
```

在所選工程根目錄執行 `build.ps1`，只建置該工程並執行本機 pre/post-build 檢查。 使用已安裝的 CS+ / CC-RH 與 PowerShell，不需額外的 Python 或 JSON 編輯環境。

工程保留所選 UART / RIIC 的 Smart Configurator component；建置使用受控的 clock、system / IRQ projection。 重新產生 SC 原始碼後，核對獨立 Log、資料腳位與時脈設定，再執行同一份建置檢查。

建置訊息

訊息	來源與處理
`W0561322` `.text` alignment mismatch	專案輸入採4-byte 對齊，原廠預編譯 C library 採2-byte。最終 `.text` 仍以4-byte 連結；保留診斷，未修改 library 或關閉警告
`W0561017` CC-RH 授權	由目前編譯環境的授權狀態產生；依正式 CC-RH 授權配置處理。不是 firmware 功能警告，也不表示零警告建置

產物	用途
Boot DefaultBuild/boot0000_FFFF.hex	燒錄器安裝 Boot
App DefaultBuild/專案名稱_update.bin	Tool 對 device 更新 App

產物	用途
App DefaultBuild/app_build_report.html	實際 BIN、CRC、metadata / IRQ 的建置結果
Transfer DefaultBuild/專案名稱.hex	燒錄器安裝獨立 Transfer 測試程式

Tool 使用 UART / Xmodem 或 I2C 的對應 Workflow。 操作步驟及訊息說明見串列 ISP 操作，欄位定義見串列工程配置。 本頁的MCU、地址、EIINT與腳位表作為RH850工程配置來源。

Metadata offset與協定配置

位址以App起點10000h、Metadata起點10100h計算；欄位offset的基準為Metadata，不是BIN起點。 CAN與串列工程分別使用其介面契約；此表為串列ISP。

Offset	Bytes	此RH850串列工程的內容 / 來源
00	4	ISP加NUL，49 53 50 00
04..07	4	format6、core128、工程flags
08..0B	4	UART protocol55h、I2C49h、Xmodem58h；版本 / checksum宣告
0C	2	target8501h；另有介面配置bytes
10 / 14 / 18	各4	App version / build / 實際image_size
1C / 20 / 24 / 28 / 2C	各4	App10000h / capacityF0000h / Flash100000h / Boot10000h / erase8000h
30 / 34 / 38 / 3C	各4	I2C編譯Hz / 7-bit address / 0 / 0；UART、Xmodem為0
40..4F / 50..5B	16 / 12	part與family ASCII、NUL padding
5C	4	最小格式長度，實際向量 / code範圍另由映像驗證器檢查
60..67	8	UPD1、policy3、32bytes、DEVICE_APP；UART / Xmodem允許UART04，I2C允許02
68 / 6C	各4	R8V1 IRQ ABI31563852h / 最低Boot版本
70 / 74	各4	UART / Xmodem編譯baud（I2C為0） / version control
78..7F / 80..FF	8 / 128	zero / FF

編輯位置為`app_image_config.h`、`image_metadata_config.h`及`isp_transport_config.h`；size與CRC由post-build產生。 例如version control位於Flash10174h；其餘位址依本表offset加10100h換算。

SC設定與重新產生

Smart Configurator 重新產生與建置

1. 保存工程設定及 App 的兩份 C header；由本工程 `.scfg` 開啟 Smart Configurator。
2. 核對 device 為 R7F701684、外部晶振16MHz。資料介面選用 Config_UART1 或 Config_RIIC1，另保留 Config_UART0、Config_OSTM0 與 Config_TAUJ0_0。
3. 執行 Generate Code，保存 SC 產生結果。既有 user-code 區保留對應 callback；資料 ISP adapter 不置於產生檔中。
4. 在該工程執行 `build.ps1`，或在已設定 hooks 的 CS+ 工程執行 Rebuild。建置會由最新 SC 來源產生受控 clock、timer / system 與完整 IRQ table，不自動重設使用者 `.scfg`。
5. 核對建置後檢查成功；App 另開啟 `app_build_report.html`，確認 metadata、CRC、512項 IRQ 與實際 BIN。App-only 變更不重建 Boot。
6. 上板時確認資料傳輸、診斷 Log、tick、中斷及重啟；離線建置通過不代表這些硬體動作已驗證。

可編輯來源	建置衍生結果	核對重點
`.scfg` / SC user-code 區	`src/smc_gen`	選定周邊與 pin 設定
SC clock / system / timer 來源	`generated_projection` 的受控 C 檔	Boot 時脈設定、App 繼承檢查、timer ISR 無重複 channel 所有權
`app_irq_handlers.h`	完整512-entry IRQ table	使用 EIINT channel；編譯後目的地與 C header 相符
`app_image_config.h` / 共用工程契約	identity、metadata、更新 BIN、HTML	編輯值與實際連結映像一致

Framed control與INFO

Frame byte	Bytes	意義
0..1	2	49 53 marker
2 / 3	各1	framing version1 / request0或response1
4..5	2	sequence LE16，1..65535
6 / 7	各1	command / status，request status0
8..9	2	payload長度LE16，0..96
10..	0..96	payload
payload之後	4	header與payload的CRC32，LE32

CRC32/ISO-HDLC：reflected polynomial EDB88320，initial / final XOR FFFFFFFF；123456789=CBF43926。 Frame最長110bytes；UART partial frame200ms逾時。相同sequence僅允許完全相同request重播。

Command	作用
01 INFO	88-byte live contract / mode / MCU / geometry / installed App / capabilities
02 ENTER	回覆完成後reset到Boot；Boot直接ACK
03 METADATA	LE16 offset加4..64bytes，依序stage256，不擦Flash
04 START / 0E FORCE_START	LE32完整size加LE32末端CRC；先驗證metadata與policy
05 WRITE / 06 END	UART / I2C：ordered64-byte write與完整映像驗證；Xmodem Boot拒絕，改由raw block / EOT處理
07 ABORT	清session / staging，不復原已擦寫內容
08 READ_METADATA / 09 VERIFY	有效App metadata讀回 / Boot唯讀重算完整映像
20 PING / 21 ECHO	App / Transfer回傳0..96bytes
22 STATS / 23 CLEAR_STATS	App / Transfer四個LE32：requests、replays、errors、written
24 STREAM_TEST	Xmodem App / Transfer宣告size1..65536與CRC32，轉到RAM診斷sink
25 DIAGNOSTICS	宣告能力的UART / Xmodem App / Transfer，回覆唯讀24-byte UART快照

以下 offset 以 INFO payload byte 0 為基準，多 byte 整數使用 little-endian。

INFO payload：88 bytes

Offset	Bytes	欄位與意義
00	1	contract：UART = 1、I2C = 2、Xmodem = 3
01	1	mode：Boot = 0、App = 1、Transfer = 2
02	1	transport：I2C = 02、UART / Xmodem = 04
03	1	max-write：UART / I2C = 64、Xmodem = 128
04..05	2	MCU target ID
06 / 07	各1	installed App valid / 更新 active
08..0B	4	App 起點
0C..0F	4	App 完整 BIN 容量
10..13	4	Code Flash bytes
14..17	4	Boot 保留 bytes
18..1B	4	Flash erase bytes
1C..1F / 20..23	各4	Boot version / build
24..27	4	編譯 bus rate：UART baud 或 I2C Hz
28..2B / 2C..2F / 30..33	各4	installed App version / build / 完整 BIN size
34 / 35	各1	installed version-control policy / App ready
36..37	2	capabilities bitmap
38..47	16	part ASCII，NUL padding
48..53	12	family ASCII，NUL padding
54..57	4	IRQ ABI

Boot能力0F、App3C、Transfer10；Transfer不回報installed App / Boot分區與IRQ ABI，其對應值為零。 提供UART診斷callback的App / Transfer另加能力bit6（0x40）；Boot不加此位元。

更新命令 payload

Command	Payload offset	Bytes	意義
03 METADATA	00..01	2	Metadata offset，4-byte 對齊
03 METADATA	02..	4..64	Metadata bytes；每段為4的倍數，依序涵蓋256 bytes
04 START / 0E FORCE_START	00..03	4	完整 BIN bytes，包含向量、Metadata、padding 與末端 CRC
04 START / 0E FORCE_START	04..07	4	BIN 末端保存的 CRC32 值
05 WRITE	00..03	4	從 BIN 起點計算的 offset
05 WRITE	04..	4..64	UART / I2C 本次寫入 bytes；非末塊64 bytes，末塊使用實際剩餘長度
06 END / 07 ABORT / 09 VERIFY	無	0	END 完整驗證、ABORT 清交易、VERIFY 唯讀驗證
08 READ_METADATA	00..01 / 02	2 / 1	Metadata offset / 要求 bytes；須為有效 installed App
24 STREAM_TEST	00..03 / 04..07	各4	Xmodem App / Transfer 的 useful size / CRC32

成功回覆保留 request 的 sequence 與 command，type = response、status = 00；有資料時依命令附 payload。 可解碼但被拒絕的命令回覆錯誤 status 與空 payload。

Status 與錯誤處理

Status	名稱	作用與核對方向
00	OK	此命令已接受；更新完成仍需 END / EOT 驗證及 App 確認
01	BAD_COMMAND	命令不屬於目前角色、協定或能力
02	BAD_SIZE	payload、offset 對齊或資料區塊長度不符合條件
03	BAD_STATE	尚未完成 Metadata staging、未 START 或目前狀態不接受命令
04	BAD_SEQUENCE	命令 sequence / 重播內容或 Metadata / BIN offset 不符合順序
05	BAD_METADATA	MCU、介面、配置、size、IRQ 或映像契約不符合
06	BAD_VERSION	有效 installed App 啟用版控，且 incoming version 較低
07	FLASH_ERROR	實際擦除或寫入 API 失敗
08	CRC_ERROR	完整映像或 VERIFY 檢查未通過

無法形成有效 frame 的資料不執行命令，也不保證有 status 回覆。 Status 名稱與值來自工程內 `hal/isp/isp_protocol.h`，命令條件由 `isp_device.c` 核對。

各介面 Bootloader 與協定格式

介面	Boot 流程	Protocol Format
UART	接收、中斷與更新	UART frame 與診斷
I2C	Slave mailbox 與更新	I2C request / reply window
Xmodem	Control / raw block 與完成	XMODEM-CRC128

UART / I2C 中斷與完成邊界

平台實作	中斷與資料處理	完成條件與說明
RLIN31 UART	RX IRQ 只放入512-slot RAM ring（511 bytes可用）；滿時丟棄新資料並計數。Error IRQ 清除並記錄錯誤	主迴圈處理 protocol / CRC / Flash / Log；TX 完整送完才執行 pending action。見UART流程
RIIC1 I2C	Event IRQ 優先於 byte IRQ；STOP 與最後 RX 同時待處理時先收該 byte。NACK 保留至 STOP，期間遮罩 NACK IRQ	完成計數扣除 DRT 預載；未完成 ready read 不 reset。見I2C格式
Xmodem	與資料 UART 共用接收 queue，framed / raw parser 分階段使用	最終 ACK 完整送完才可 reset；App / Transfer使用RAM sink。見Xmodem流程

App 的 ready 回覆須已觀察到系統 tick；Transfer 不宣告已安裝 App 或 Flash 分區。 串列診斷欄位集中於UART診斷格式。 RIIC 暫存器與交易順序依 RH850/F1KH、F1KM 硬體手冊 第23章核對，特別是23.3.15、23.4、23.5.5。

驗證範圍

離線檢查與實板範圍

`test_rh850_isp.ps1` 使用實際 App BIN、C 引擎 / Flash page adapter / 映像驗證器及 Tool client，核對完整960KiB、512-entry table、CRC / replay / 失敗後不得 reset 等條件。 `test_rh850_isp_runtime.ps1` 直接編譯實際 `RuntimePoll`、角色設定與 Flash / 串流 adapter，分別測試三介面的 Boot / App / Transfer。 測試包含分段 / 損壞封包、精確重送、ready、角色權限、最終回覆完成及傳送失敗；Xmodem 診斷另涵蓋短尾、block wrap、CRC16 重試、完整 CRC32、取消與 timeout。 `test_rh850_isp_hal.ps1` 使用實際 ISR adapter 與模擬暫存器，核對 queue / NACK / STOP / 短讀；它不模擬電氣波形與硬體時序。

`test_rh850_scfg_roundtrip.ps1` 在隔離副本呼叫已安裝的 Smart Configurator 重新產生原始碼，再建置及核對映像 / RAM / IRQ；不以手工複製來源取代 SC 產生動作。 完成後比較原工程來源與 App-only Boot 交付檔雜湊，將設定檢查、實際產生、連結結果及硬體結果分開記錄。

實板後續核對 UART、Xmodem、I2C 的 Discover、Transfer、Enter Boot、App 更新與重連。 另外確認 FCL RAM 執行、獨立 Log、IRQ 持續運作、中斷 / 斷電與復原行為。

UART Bootloader 與協定格式

本章使用 framed UART ISP；每份 Boot、App、Transfer 各自是獨立工程。 UART 資料與診斷 Log 分開接線；Tool 的 Host Baud 與 Device baud 也分別設定。

工程與接線

用途	工程或連線	詳細說明
Boot / App / Transfer	選擇名稱含 UART 的對應工程；各自包含來源與建置工具	平台工程與周邊
ISP 資料	Bridge 資料 UART TX 接 device RX，RX 接 TX；共地並核對電壓	Tool 接線與操作
診斷文字	Device 的獨立 Log UART 接 terminal	平台工程與周邊
建置與下載檔	在所選工程執行 PowerShell 建置；Boot 安裝檔與 App 更新 BIN 分別使用	編輯、產生與建置

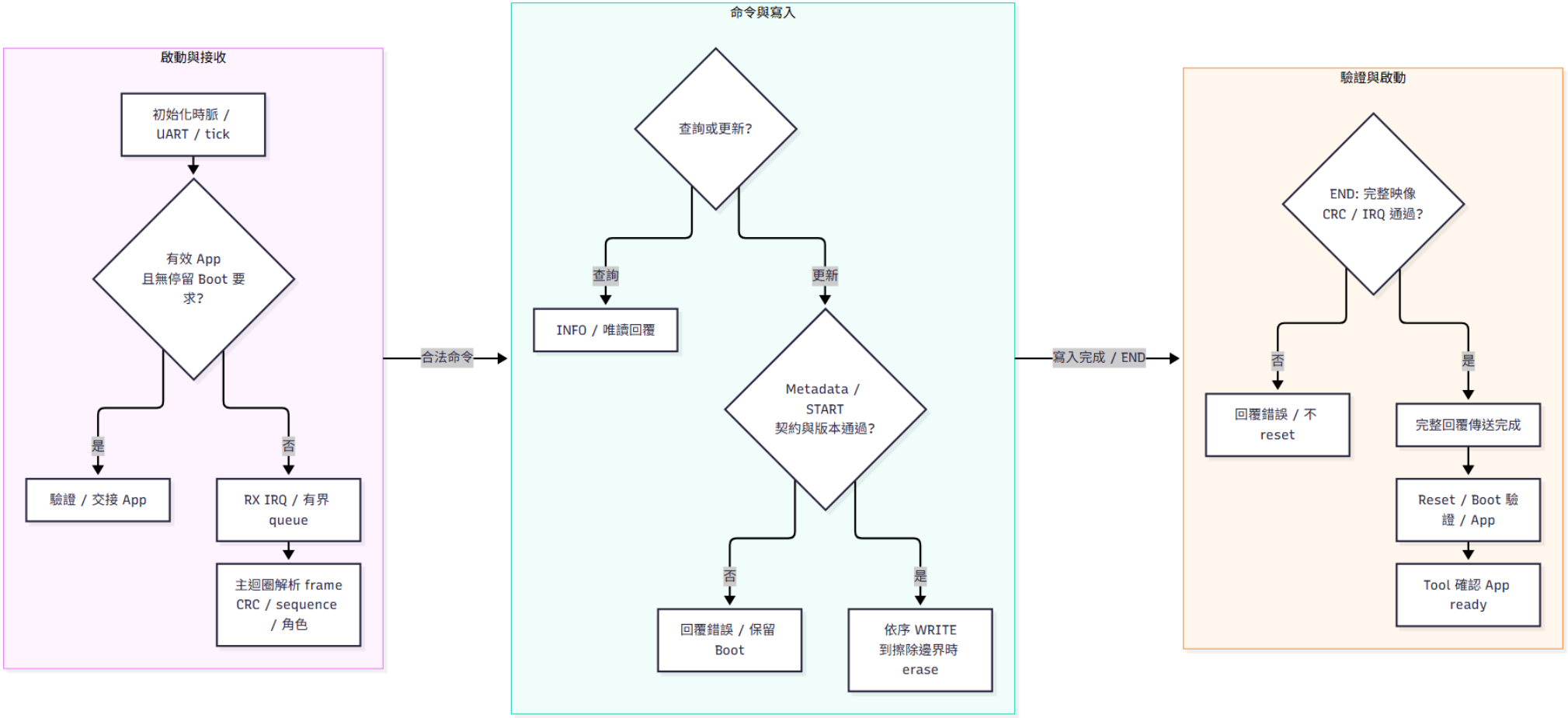
App-only 建置使用 App 自己的契約與工具；安裝到板上的 Boot 仍需與 App 相容。 Custom Transfer 是獨立通訊診斷映像，不擦寫 App Flash。

通訊設定

修改來源	作用	同步核對
`isp_transport_config.h` 的 `ISP_UART_BAUD`	Device 資料 UART 速率，範例為 115200 bit/s、8-N-1	配對 Boot / App 的周邊分頻、INFO、BIN Metadata 與 Tool Device baud
`ISP_POLICY_UART_BAUD`	映像契約宣告的 UART baud	與實際初始化及上述速率相符
`.scfg` 或各工程 board adapter	周邊、腳位、來源時脈與分頻	SC 設定與重新產生
`ISP_INTERBYTE_TIMEOUT_MS`	未完成封包的接收閒置期限，範例 200 ms	與實際傳輸及接收處理相符
`ISP_BOOT_IDLE_TIMEOUT_MS`	有效 App 進入 Boot 後、更新尚未開始時的閒置返回期限，範例 30000 ms	已開始的更新不套用此閒置返回

變更 baud 時重建配對 Boot / App，並安裝更新後的 Boot。 可選速率與必要條件集中於 通訊設定與更新必要條件。 Metadata 欄位位置見 Metadata offset 與協定配置。

Boot 執行流程



INFO 等唯讀命令也由主迴圈回覆；ISR 只處理接收資料與錯誤狀態。 Metadata staging 與 START 不擦除 Flash；第一個合法 WRITE 才依擦除邊界啟動 Flash 操作。

Protocol Format

UART 依 byte stream 傳送共用 ISP frame。接收端依 header 的 payload length 找到 CRC32 尾碼， 不以 UART idle、換行或 terminal 文字作為命令結尾。

層級	內容與長度	參考
Frame	10-byte header + 0..96-byte payload + 4-byte CRC32，最長 110 bytes	逐 byte 封包與命令表

層級	內容與長度	參考
METADATA 03	LE16 Metadata offset + 4..64 bytes；依序暫存全部 256 bytes	Offset 基準是 Metadata 起點
START 04	LE32 完整 BIN bytes + LE32 映像末端 CRC32	驗證已暫存 Metadata、配置與版本政策
WRITE 05	LE32 BIN offset + 最多 64 bytes；非末塊為 64 bytes	Offset 基準是 BIN 起點，實體位址 = App 起點 + offset
END 06	空 payload	重新驗證完整已寫入映像，再回覆結果

CRC32 有兩個用途：frame 尾碼保護本次 header / payload；BIN 末端 4 bytes 保護映像中尾碼之前的內容。 兩者分別計算，不能以封包 CRC 正確推定 Flash 映像有效。

完整封包範例

以下 INFO request 為 sequence = 1、command = 01、空 payload；可以送給 Boot、App 或 Transfer。 最後四個 bytes 是前十個 bytes 的 CRC32，採 little-endian。

```
49 53 01 00 01 00 01 00 00 00 F1 8D B0 75
```

具體回覆的 88-byte INFO payload 取決於板上角色、MCU、分區與 installed App；欄位定義見共用契約。

成功與錯誤回覆範例

下列為依格式計算的封包範例，不是板端擷取記錄。以Transfer角色為例，PING將payload原樣回覆；未知命令回覆`BAD_COMMAND=1`，兩者的sequence / command都對應request。

```
PING request: 49 53 01 00 02 00 20 00 03 00 12 34 56 1F 91 6C 3F
PING reply:   49 53 01 01 02 00 20 00 03 00 12 34 56 21 FA AE D0
Unknown request: 49 53 01 00 03 00 7F 00 00 00 55 FB 98 B3
Error reply:     49 53 01 01 03 00 7F 01 00 00 D6 9A 2D 14
```

Header的response byte由00改為01；status為00成功、01未知命令。

每列最後4bytes分別對該列header與payload計算CRC32；錯誤回覆也有自己的有效CRC。

異常與完成條件

觀察	Device / Tool 行為	核對方向
Frame 未收齊或 CRC 錯誤	不執行該命令；partial frame 超過設定期限後清除	baud、接線、queue / 硬體錯誤、封包長度
同一 sequence 重送	只接受完整相同 request，重播快取回覆	不把重播當作第二次寫入
START / WRITE 被拒絕	回覆對應 status，不以錯誤 request 授權 Flash 操作	狀態碼與命令條件
END 驗證成功	完整回覆傳送完成後才 reset	Device 傳送完成與 Tool 收到回覆分別判斷
App 確認尚未成功	保留報告並讀取即時狀態	不自動重複 erase / END / RUN 來推定結果

實際操作與報告判讀見 Tool 串列操作。 平台與實板驗證範圍見 驗證範圍。

程式對照與診斷

程式位置	入口或 API	責任
`custom_func.c`	`hardware_init()` / `loop()`	平台初始化、封包處理及回覆完成後的交接順序
`hal/isp/isp_protocol.c`	`IspStreamFeed()` / `IspEncode()`	串流組包、長度與 frame CRC32
`hal/isp/isp_device.c`	`IspDeviceProcess()` / `IspDeviceWrite()` / `IspDeviceFinish()`	命令、角色、版本、範圍及更新狀態
`hal/device`	各平台 UART RX / Error ISR、board API	有界 queue、周邊初始化與傳送完成
`hal/image` / `hal/flash`	映像驗證與 Flash API	Metadata、完整 CRC、IRQ 與實際擦寫

UART診斷格式

Command0x25不帶payload。只有INFO宣告bit6時才查詢；回覆是下表24bytes，多byte整數採little-endian。 回覆使用相同sequence快取規則；重播同一request不重新讀取硬體。

Offset	Bytes	內容與範例
00	1	格式1
01	1	診斷種類1：Device UART
02..03	2	欄位mask0x000F，代表本表四組計數 / queue欄位均提供
04..07	4	RX硬體錯誤次數
08..0B	4	RX queue丟棄新資料次數
0C..0F	4	本機TX timeout次數
10..13	4	RX queue最高使用bytes
14..17	4	快照當下RX queue待處理bytes

RH850 硬體錯誤 / 丟棄 / TX 計數飽和於 0xFFFFFFFF，欄位統一序列化為 32 bits。 最高量與待處理量是bytes，不是CAN frame數；reset後重新起算，不宣告跨重啟的累積量。 快照只短暫保留並還原對應IRQ遮罩，不清空接收queue或重設周邊。 `CLEAR_STATS`只清除協定計數，不清除此硬體診斷快照的累積量。 Tool保留支援 / 未提供 / 讀取失敗的區別；畫面與報告的用途見Device串列診斷。

I2C Bootloader 與協定格式

Device 使用 I2C Slave，bridge 使用 I2C Master。每份 Boot、App、Transfer 各自是獨立工程， 另保留獨立 UART 診斷文字。

工程與接線

用途	工程或連線	詳細說明
Boot / App	選擇名稱含 ISP_Boot_loader_I2C 的配對工程	平台工程與周邊
Custom Transfer	名稱含 I2C_Slave_Transfer 的獨立診斷工程	透過相同 mailbox 驗證 PING / ECHO / 計數，不擦 Flash
I2C 資料	SCL、SDA、共地；核對上拉電壓、阻值與負載	Tool 接線與操作
診斷文字與產物	獨立 Log UART；Boot 安裝檔 / App 更新 BIN 分開使用	編輯、產生與建置

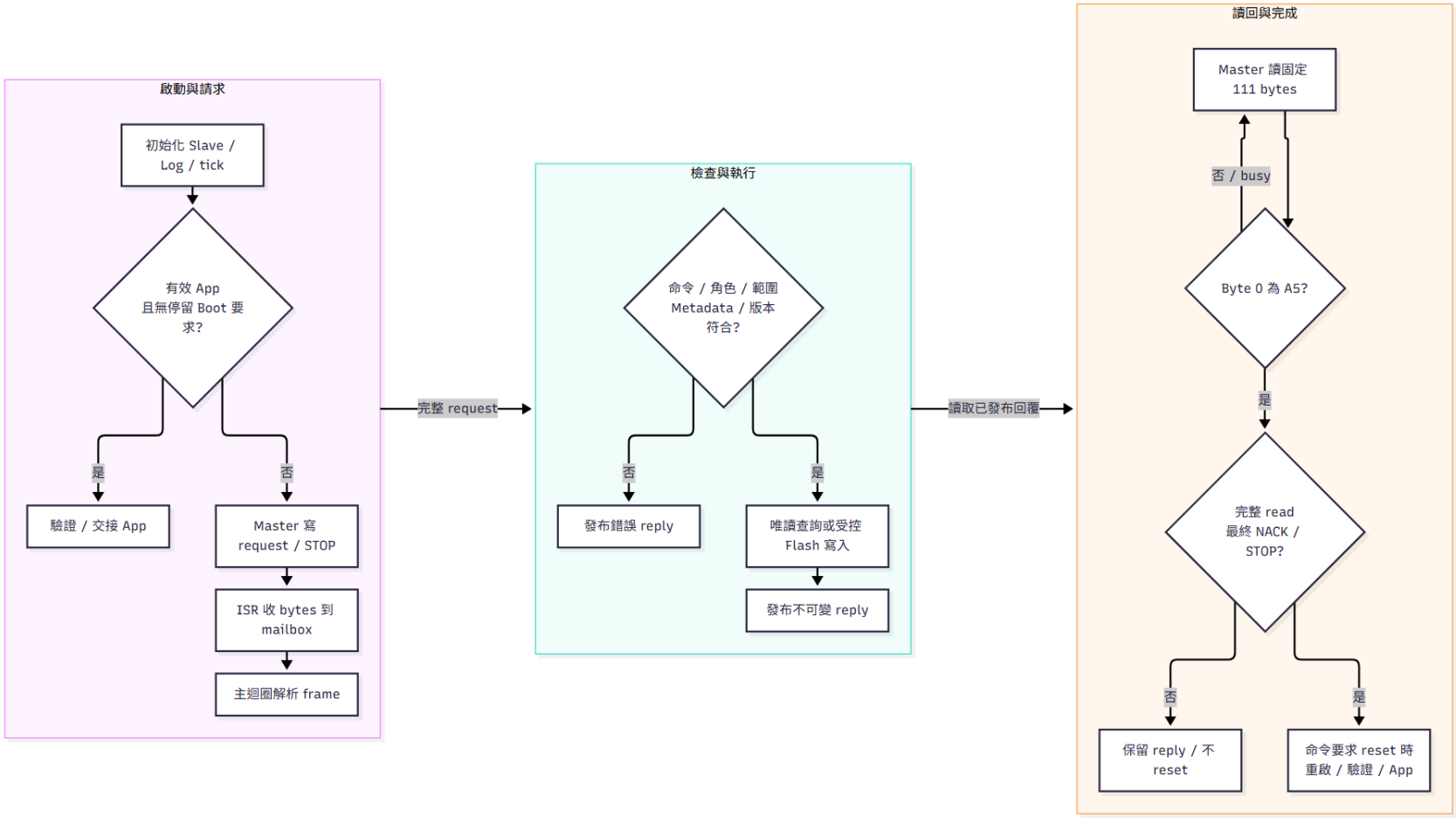
各工程包含自身建置來源，不需要其他工程 checkout 才能編譯。 燒錄後配對的 Boot / App 保持相同介面及映像契約。

通訊設定

修改來源	作用	同步核對
`ISP_I2C_ADDRESS` / `ISP_LINK_PARAMETER_1`	7-bit Slave address，範例 0x20	Boot / App slave 初始化、BIN Metadata + 0x34 與 Tool 所選位址
`ISP_BUS_RATE` / `ISP_LINK_PARAMETER_0`	編譯 bus rate，範例 100000 Hz	Slave 時序、INFO、BIN Metadata + 0x30 與 Tool 100 / 400 kHz 能力
`.scfg` 或各工程 board adapter	RIIC、腳位、時脈與中斷	平台配置、SC 設定
`ISP_BOOT_IDLE_TIMEOUT_MS`	有效 App 進入 Boot 後、更新尚未開始時的閒置返回期限	範例 30000 ms；active 更新不套用此條件

Tool 輸入 7-bit 位址 0x20，不是 address byte 0x40。 完整設定條件集中於 通訊設定與更新必要條件。 INFO 不含 Slave address；Tool 使用實際交易位址，並核對讀回的 Metadata。

Boot 執行流程



START 檢查完整 staged Metadata；WRITE 依序寫入，END 驗證完整映像。 I2C ACK 只代表 bus byte / 交易的接收，更新結果仍以 framed reply status 與 App 確認判斷。

Protocol Format

I2C 先寫入一份以 STOP 分隔的完整 request，再讀取固定 111-byte reply window。 ISR 只處理 bytes 與 START / NACK / STOP；主迴圈才解析、執行 Flash 並發布不可變回覆。

交易 / byte	長度與內容	意義
Write	START + address/W + 完整 ISP frame + STOP	Frame 為 10-byte header + 0..96 payload + CRC32，最長 110 bytes

交易 / byte	長度與內容	意義
Read byte 0	00 busy 或 A5 ready	只有 A5 後才解析 framed reply
Read byte 1..N	完整 framed reply，14..110 bytes	sequence / command 對應 request，status 回報命令結果
Read 剩餘 bytes	FF padding，直到總計 111 bytes	由內層 frame length 決定有效資料，不把 padding 納入 frame CRC
Read 結尾	Master 最終 NACK，接著 STOP	與完整 read 一起作為回覆完成條件

完整 header、各命令 payload、INFO 與 status 定義見 共用封包與命令表。 UART / I2C 使用相同命令格式，I2C 外層 mailbox byte 不屬於內層 CRC 範圍。

位址與交易範例

7-bit address = 0x20 時，線上 address byte 為寫入 0x40、讀取 0x41。 以下以 INFO request 為例；[] 表示說明佔位，不是線上 bytes。

```
S 40 49 53 01 00 01 00 01 00 00 00 F1 8D B0 75 P
S 41 00 [busy read window] NACK P
S 41 A5 [   INFO framed reply] [FF padding] NACK P
```

S 為 START，P 為 STOP；每次 read window 都讀取 111 bytes。 busy 後只重讀 mailbox，不重新送出前一個 Flash request。

成功與錯誤回覆範例

I2C內層沿用 UART完整request / reply範例。 以PING為例，ready window為A5 + 上述PING reply + FF padding至111bytes；未知命令則為A5 + error reply + padding。 A5只表示已有完整回覆；更新是否獲接受仍看內層status。 busy window的首byte為00，不是BAD_STATE或CRC錯誤回覆。

異常與完成條件

觀察	Device / Tool 行為	核對方向
無 address ACK	尚無有效 ISP 回覆	7-bit 位址、接線、電壓、上拉、Slave 初始化
Byte 0 = 00	唯讀 poll 等待 ready	不因 busy 重送 Flash request
Short read	保留 reply；不執行 pending reset	固定 111-byte read、最終 NACK 與 STOP
CRC / sequence / 命令錯誤	不以錯誤交易授權 Flash；可解碼命令回報對應 status	共用狀態碼
回覆已完整讀取	僅已接受命令的 pending action 可執行	ENTER / END 的重啟條件與 Tool App 確認

mailbox 的交易 lease 為 2 秒；逾時清除 mailbox 狀態不表示恢復已擦寫的 App。 更新 / 取消與報告見 Tool 串列操作。 平台與實板驗證範圍見 驗證範圍。

程式對照與診斷

程式位置	入口或 API	責任
`custom_func.c`	`hardware_init()` / `loop()`	初始化、主迴圈命令處理與 pending action
`hal/isp/isp_i2c_mailbox.c`	`IspMailboxStart()` / `IspMailboxReceive()` / `IspMailboxNext()` / `IspMailboxNack()` / `IspMailboxStop()`	有界交易資料與狀態，不解析 ISP 命令
同一 mailbox 模組	`IspMailboxGet()` / `Publish()` / `Release()`	主迴圈取 request、發布及釋放 reply
`hal/isp/isp_protocol.c` / `isp_device.c`	`IspStreamFeed()` / `IspDeviceProcess()`	內層 frame 與更新命令
`hal/device` 與各平台 I2C adapter	RIIC ISR、reply-complete 判斷	暫存器、STOP / NACK / 最後資料的完成邊界

各平台所用 peripheral、實體向量 / EIINT 與 Flash API 集中於 平台配置。 Metadata、IRQ 與 Flash 驗證維持各平台工程的契約。

Xmodem Bootloader 與協定格式

本章使用 framed ISP control 搭配 XMODEM-CRC / 128-byte block。 Tool 在 **UART / Xmodem → Workflow** 選擇對應功能；Device 使用獨立 Xmodem 工程，另保留 Log UART。

工程與接線

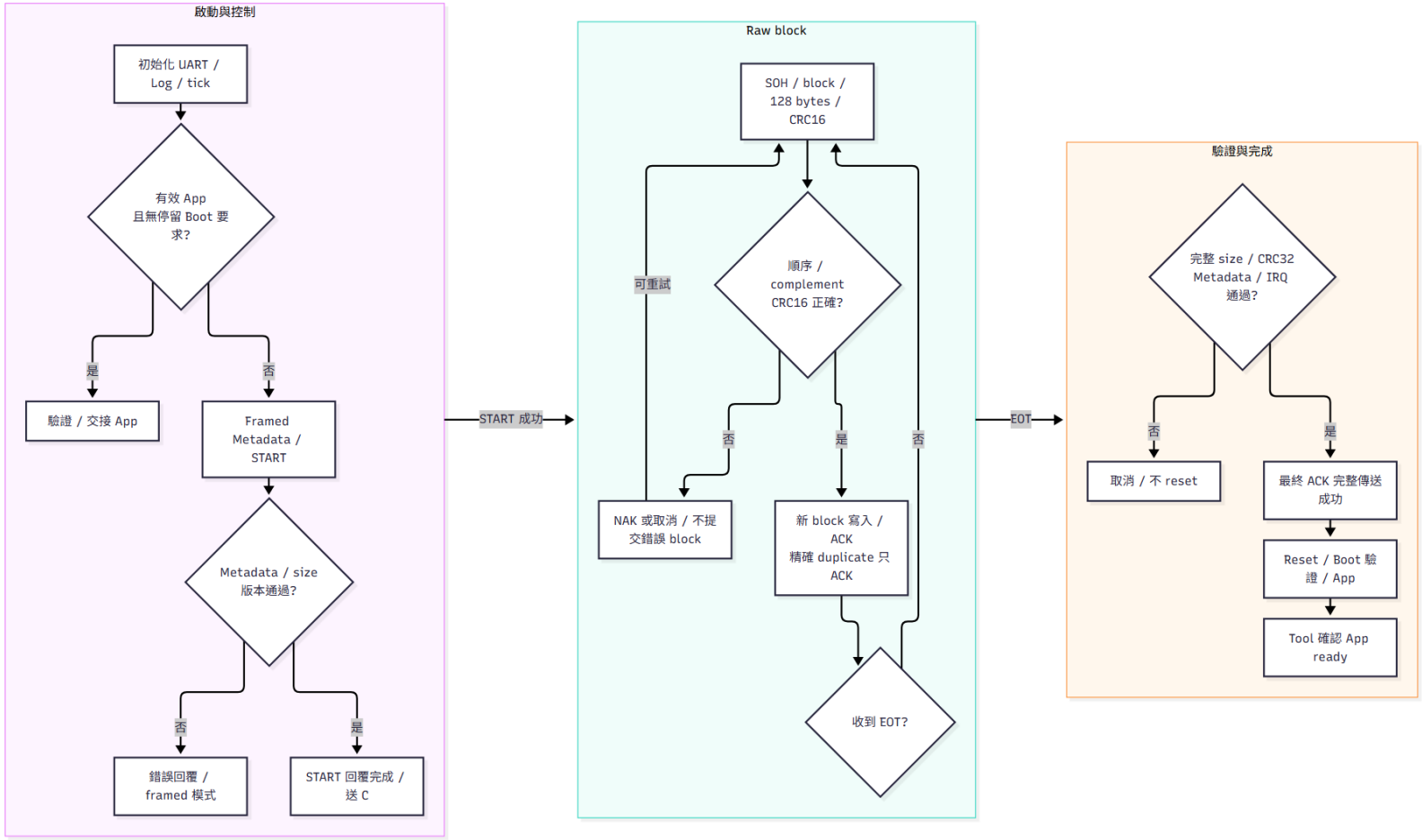
用途	工程或連線	詳細說明
Boot / App / Transfer	選擇名稱含 Xmodem 的對應工程，各自可獨立建置	平台工程與周邊
資料與文字	資料 UART 與獨立 Log UART 分別接線	Tool 接線與操作
App 更新	Tool 載入封裝的 `_update.bin`，以 XMODEM-CRC Workflow 更新	建置產物
Transfer 診斷	使用 STREAM_TEST 接收 RAM CRC32 sink	不擦 Flash；不回報已安裝 App 的成功更新

通訊設定

修改來源或條件	作用	同步核對
`ISP_UART_BAUD` / `ISP_POLICY_UART_BAUD`	資料 UART 速率，範例 115200 bit/s、8-N-1	配對 Boot / App 分頻、INFO、Metadata 與 Tool Device baud
`.scfg` 或 board adapter	UART、pin、clock 與接收中斷	SC 設定與重新產生
XMODEM-CRC128	固定 128-byte block，加 CRC16/XMODEM	與 Tool 的 XMODEM-CRC Workflow 相符
`ISP_BOOT_IDLE_TIMEOUT_MS`	更新尚未開始時、有效 App 的 Boot 閒置窗口	範例 30000 ms；raw streaming 另有接收期限

Metadata 的 protocol 為 Xmodem；其 update transport 為 UART。 資料腳位相同不代表 UART framed WRITE 可取代 Xmodem raw block。 設定條件見 通訊設定與更新必要條件。

Boot 執行流程



Boot 更新以 staged Metadata 與完整 BIN size 綁定串流，排除末塊 padding。 App / Transfer 的 STREAM_TEST 使用同一 block 接收方式，但資料只送入 RAM CRC32 sink。

Protocol Format

Byte / 控制	意義
0	SOH01
1 / 2	8-bit block number / ones complement，可wrap
3..130	128-byte payload，末塊unused positions填1A
131 / 132	CRC16/XMODEM high / low byte，poly1021、init0
C / ACK / NAK / CAN	43 / 06 / 15 / 18；兩個CAN取消
EOT04	完整映像或診斷CRC32 finalization，只送一次

階段	線上格式	判斷依據
準備	共用 ISP framed INFO / ENTER / METADATA / START	共用 header、命令與 INFO
Raw data	SOH、block number、complement、128 bytes、CRC16，共 133 bytes	CRC16 只覆蓋 128-byte payload，尾碼 high byte 先送
末塊	不足 128 bytes 的位置填 1A	只提交 START 宣告的 useful bytes
結束	EOT 與最終 ACK	Boot 完整 BIN CRC32 / Metadata / IRQ 驗證通過後才完成

CRC16/XMODEM 使用 polynomial 1021h、initial 0000h、無反射、無 final XOR；`123456789` 的結果為 31C3h。 它保護單一 block；共用控制 frame 與完整映像仍各自使用 CRC32。

Block 範例

```
01 01 FE [payload: 00 01 02 ... 7F   128 bytes] E8 0A
```

第一個 01 是 SOH，第二個 01 是 block number，FE 是其 ones complement。 E8 0A 為這組 128 bytes 的 CRC16 high / low byte；block number 以 8-bit wrap，從 FF 接到 00。

成功與錯誤回覆範例

Request / 狀態	Device回覆	意義
上述block通過CRC、順序與寫入	06 ACK	可以送下一block；完整映像仍待EOT驗證
CRC錯誤或partial block超時	15 NAK	重送相同block；不跳到下一block
完整相同duplicate	06 ACK	重播接受結果，不再次寫入
EOT且完整映像驗證通過	06最終ACK，傳送完成後reset	Tool另行確認App ready
取消或完整映像驗證失敗	18 18 CAN CAN	不以ACK或重啟推定成功；保存結果後查詢狀態

ACK / NAK是raw單byte控制，不含ISP frame CRC；framed錯誤回覆只在framed控制階段使用。

異常與完成條件

條件	處理	完成判斷
Block CRC16 / 順序錯誤	NAK；Tool 對相同 block 最多三次嘗試	尚未提交錯誤 block
精確 duplicate block	全部 128-byte 內容相同才重播 ACK	不重複寫入 Flash
Partial block 閒置 200 ms	NAK 並清除 partial block	等待合法 block
Streaming idle 10 秒或收到兩個 CAN	取消串流，返回 framed parser	不執行 pending reset
本機 C / ACK / NAK 傳送失敗	取消串流並清除 pending reset	不以未完成回覆授權重啟
EOT 完整驗證成功	最終 ACK 完整傳送後才 reset	Tool 另行確認重連與 App ready

EOT 只送一次；最終 ACK 遺失時保留映像與報告，再查詢即時狀態，不以重送 EOT 或重燒推定結果。 平台與實板驗證範圍見 驗證範圍。

程式對照與診斷

程式位置	入口或 API	責任
`custom_func.c`	`hardware_init()` / `loop()`、串流 callback	framed / raw 模式切換、sink 與回覆完成後動作
`hal/isp/isp_protocol.c` / `isp_device.c`	`IspStreamFeed()` / `IspDeviceProcess()`	Metadata、START、版本與角色契約
`hal/isp/isp_xmodem.c`	`IspXmodemInit()` / `IspXmodemFeed()` / `IspXmodemPoll()`	block、CRC16、duplicate、cancel 與接收期限
同一 Xmodem 模組	`IspXmodemCrc()`	CRC16/XMODEM
`hal/image` / `hal/flash`	映像驗證與 Flash API	Boot 寫入及完整 BIN 驗證；診斷 sink 不使用 Flash

Tool 的 Xmodem Transfer 使用 389 useful bytes，涵蓋末塊填充及 CRC32。 UART queue 與硬體錯誤欄位沿用 UART 診斷格式。 Tool 操作與報告集中於 串列操作。